

```

In [ ]: class ZeidelSolver:
    def __init__(self, A, b, eps=1e-4):
        self.A = [row[:] for row in A]
        self.b = b[:]
        self.eps = eps
        self.n = len(b)

    def make_diagonally_dominant(self):
        """Еквівалентні перетворення системи для діагональної
        переваги"""
        # 1. Міняємо місцями 4-й і 5-й рядки (індекси 3 та 4)
        self.A[3], self.A[4] = self.A[4], self.A[3]
        self.b[3], self.b[4] = self.b[4], self.b[3]

        # 2. Модифікуємо 2-й рядок (індекс 1) за допомогою
        початкового 3-го (індекс 2)
        # Копіюємо початковий 3-й рядок перед його зміною
        original_R3 = self.A[2][:]
        original_b3 = self.b[2]

        self.A[1] = [a - 0.3 * c for a, c in zip(self.A[1],
        original_R3)]
        self.b[1] = self.b[1] - 0.3 * original_b3

        # 3. Модифікуємо 3-й рядок (індекс 2) за допомогою нового
        4-го (індекс 3)
        self.A[2] = [a - 0.35 * c for a, c in zip(self.A[2],
        self.A[3])]
        self.b[2] = self.b[2] - 0.35 * self.b[3]

    def print_system(self, title):
        print(f"\n=== {title} ===")
        for row, val in zip(self.A, self.b):
            row_str = " | ".join(f"{x:10.6f}" for x in row)
            print(f"[{row_str}] = {val:10.6f}")

    def solve(self, max_iter=100):
        x = [0.0] * self.n
        print(f"\n=== Розв'язання методом Зейделя (eps =
        {self.eps}) ===")

        for iteration in range(1, max_iter + 1):
            x_new = x[:]
            for i in range(self.n):
                # Сума для вже знайдених (нових) значень
                s1 = sum(self.A[i][j] * x_new[j] for j in
                range(i))

                # Сума для значень з попередньої ітерації
                s2 = sum(self.A[i][j] * x[j] for j in range(i + 1,
                self.n))

                x_new[i] = (self.b[i] - s1 - s2) / self.A[i][i]

            # Обчислення вектора r = |b - Ax| для поточної

```

```

ітерації
        residuals = []
        for i in range(self.n):
            ax = sum(self.A[i][j] * x_new[j] for j in
range(self.n))
            residuals.append(abs(self.b[i] - ax))

        # Перевірка умови зупинки:  $\max |x_{\text{new}} - x_{\text{old}}| < \text{eps}$ 
        max_diff = max(abs(x_new[i] - x[i]) for i in
range(self.n))

        # Форматування виводу ітерації
        x_str = ", ".join(f"{val:9.6f}" for val in x_new)
        r_str = ", ".join(f"{r:9.2e}" for r in residuals)
        print(f"Ітерація {iteration}:")
        print(f"  x = [{x_str}]")
        print(f"  r = [{r_str}]")

        x = x_new
        if max_diff < self.eps:
            print(f"\nДосягнуто заданої точності на
{iteration}-й ітерації.")
            break
        return x

```

```

In [2]: A_orig = [
        [7.03, 1.22, 0.85, 1.135, -0.81],
        [0.98, 3.39, 1.30, -1.63, 0.57],
        [1.09, -2.46, 6.21, 2.10, 1.033],
        [1.345, 0.16, 2.10, 5.33, -12.00],
        [1.29, -1.23, -0.767, 6.00, 1.00]
        ]
        b_orig = [2.1, 0.84, 2.58, 11.96, -1.47]

```

```
In [3]: solver = ZeidelSolver(A_orig, b_orig, eps=1e-4)
solver.print_system("Початкова матриця")

solver.make_diagonally_dominant()
solver.print_system("Матриця після забезпечення діагональної
преваги")

solution = solver.solve()
```

=== Початкова матриця ===

```
[ 7.030000 | 1.220000 | 0.850000 | 1.135000 |
-0.810000] = 2.100000
[ 0.980000 | 3.390000 | 1.300000 | -1.630000 |
0.570000] = 0.840000
[ 1.090000 | -2.460000 | 6.210000 | 2.100000 |
1.033000] = 2.580000
[ 1.345000 | 0.160000 | 2.100000 | 5.330000 |
-12.000000] = 11.960000
[ 1.290000 | -1.230000 | -0.767000 | 6.000000 |
1.000000] = -1.470000
```

=== Матриця після забезпечення діагональної переваги ===

```
[ 7.030000 | 1.220000 | 0.850000 | 1.135000 |
-0.810000] = 2.100000
[ 0.653000 | 4.128000 | -0.563000 | -2.260000 |
0.260100] = 0.066000
[ 0.638500 | -2.029500 | 6.478450 | 0.000000 |
0.683000] = 3.094500
[ 1.290000 | -1.230000 | -0.767000 | 6.000000 |
1.000000] = -1.470000
[ 1.345000 | 0.160000 | 2.100000 | 5.330000 |
-12.000000] = 11.960000
```

=== Розв'язання методом Зейделя (eps = 0.0001) ===

Ітерація 1:

```
x = [ 0.298720, -0.031266, 0.438425, -0.259589, -1.002178]
r = [ 8.52e-01, 7.92e-02, 6.84e-01, 1.00e+00, 1.78e-15]
```

Ітерація 2:

```
x = [ 0.177575, -0.031281, 0.556016, -0.051484, -0.902745]
r = [ 2.56e-01, 5.11e-01, 6.79e-02, 9.94e-02, 0.00e+00]
```

Ітерація 3:

```
x = [ 0.141218, 0.098176, 0.589671, -0.029398, -0.889395]
r = [ 2.01e-01, 6.54e-02, 9.12e-03, 1.34e-02, 0.00e+00]
```

Ітерація 4:

```
x = [ 0.112655, 0.118535, 0.597457, -0.020314, -0.886927]
r = [ 3.98e-02, 2.43e-02, 1.69e-03, 2.47e-03, 0.00e+00]
```

Ітерація 5:

```
x = [ 0.106998, 0.125310, 0.599876, -0.017811, -0.885936]
r = [ 1.24e-02, 6.76e-03, 6.77e-04, 9.92e-04, 0.00e+00]
```

Ітерація 6:

```
x = [ 0.105240, 0.127226, 0.600545, -0.017119, -0.885683]
r = [ 3.49e-03, 1.87e-03, 1.72e-04, 2.53e-04, 0.00e+00]
```

Ітерація 7:

```
x = [ 0.104744, 0.127758, 0.600734, -0.016922, -0.885611]
r = [ 9.76e-04, 5.35e-04, 4.95e-05, 7.24e-05, 0.00e+00]
```

Ітерація 8:

```
x = [ 0.104605, 0.127910, 0.600788, -0.016866, -0.885590]
```

```
r = [ 2.77e-04, 1.51e-04, 1.40e-05, 2.05e-05, 0.00e+00]
```

Ітерація 9:

```
x = [ 0.104566, 0.127952, 0.600803, -0.016850, -0.885584]
```

```
r = [ 7.81e-05, 4.25e-05, 3.95e-06, 5.78e-06, 0.00e+00]
```

Досягнуто заданої точності на 9-й ітерації.
